

pmisetup

November 29, 2017
17:11

Contents

1	Routines used to set up the PMI file	1
2	References	4
3	INDEX	5

1 Routines used to set up the PMI file

\$Id: 229593edde6c67445687f491571c6b49c862b85a \$

The file `pmi_infile` identified in `degas2.in` describes all of the plasma-material interactions (PMI) available in DEGAS 2. Each interaction is described by four separate lines:

1. The full name of the interaction
2. A blank delimited sequence of the form
`species + material -> product1 + product2 + ...`

where “species” is a species symbol and “material” is a symbol from the list in `materials_infile`. All of the products should be legitimate species. Currently, only binary reactions of this form are supported. Essentially all PMI currently in use yield zero (e.g., an exit) or one product; support of additional products within a given PMI will require some additional code. With the “->” symbol, the PMI holds for all isotopically equivalent species; it’s a generic PMI. Non-generic PMI are specified by a “=>” symbol. For the latter, the PMI applies to *only* the species explicitly appearing in the file.

3. The path name (relative or absolute) of the netCDF data file providing the yield and PMI handling data for this interaction.
4. The PMI type. This is used to group PMI which are sufficiently similar that their products and collisions can be processed using the same code. Currently recognized types include

reflection Reflection of the test particle with a significant fraction of its incident energy.

desorption Thermal re-emission of not-permanently adsorbed test particles.

adsorption Permanent adsorption (i.e., removal of the incident test particle from the problem). Note that by implementing this process as a PMI (as opposed to varying the material’s recycling coefficient or designating a sector as being of exit type), the amount of absorption can be made to vary with species or other parameters.

```
"pmisetup.f" 1 ≡
  @m FILE 'pmisetup.web'
```

The unnamed module.

```
"pmisetup.f" 1.1 ≡
  ⟨ Functions and Subroutines 1.2 ⟩
```

Read in data from **pmi_infile**.

⟨Functions and Subroutines 1.2⟩ ≡

```

subroutine read_pmi
  implicit none f77
  pm_common    // Common
  sp_common
  ma_common
  rf_common
  implicit none f90
  integer length, p, b, e, i    // Local
  character*LINELEN line
  character*FILELEN file
  character*FILELEN tempfile    // local
  character*pm_type_len type

  ⟨Memory allocation interface 0⟩
  st_decls
  pm_num = 0

  pmi_version = 'unknown'

  tempfile = filenames_array_pmi_infile
  open(unit = diskin, file = tempfile, form = 'formatted', status = 'old')
  assert(read_string(diskin, line, length))
  assert(length ≤ len(line))
  pmi_version = line(: length)
loop1: continue
  if (¬read_string(diskin, line, length))
    goto eof    /* Add next pmi */
  pm_num = pm_num + 1
  var_realloca(pmi_name)
  var_realloca(pmi_type)
  var_realloca(pmi_sy)
  var_realloca(pmi_reagent)
  var_realloca(pmi_materials)
  var_realloca(pmi_generic)
  var_realloca(pmi_product_num)
  var_realloca(pmi_product)
  var_realloca(pmi_filename)

  /* First line contains the full name of the pmi */
  length = parse_string(line(: length))
  pmi_name_pm_num = line(: length)

  assert(read_string(diskin, line, length))
  length = parse_string(line(: length))    /* Identify reagent species and material */
  p = 0
  assert(next_token(line, b, e, p))
  pm_sy(pm_num) = read_text(line(b : e))
  assert(next_token(line, b, e, p))
  pm_reagent(pm_num) = sp_lookup(line(b : e))
  assert(next_token(line, b, e, p))
  assert(line(b : e) ≡ '+')
  assert(next_token(line, b, e, p))
  pm_materials(pm_num) = ma_lookup(line(b : e))

```

```

    assert(next_token(line, b, e, p))
    if (line(b : e) ≡ '->') then
        pm_gen(pm_num) = pmi_generic_yes    // Use generic species
    else if (line(b : e) ≡ '=>') then
        pm_gen(pm_num) = pmi_generic_no    // Specific species
    else
        assert('Invalid_token' ≡ '␣')
    end if    /* Count and identify product species */
    pm_product_num(pm_num) = 0
    do i = 1, pm_product_max
        pm_product(pm_num, i) = int_unused
    end do
loop2: continue
    if (next_token(line, b, e, p)) then
        pm_product_num(pm_num) = pm_product_num(pm_num) + 1
        assert(pm_product_num(pm_num) ≤ pm_product_max)
        pm_product(pm_num, pm_product_num(pm_num)) = sp_lookup(line(b : e))
        assert(pm_product(pm_num, pm_product_num(pm_num)) > 0)
        if (next_token(line, b, e, p)) then
            assert(line(b : e) ≡ '+')
            goto loop2
        end if
    end if    /* Third line contains the path name for the data file */
    assert(read_string(diskin, file, length))
    assert(length ≤ len(file))

    pm_filename(pm_num) = file

    /* Fourth line gets the interaction type */
    assert(read_string(diskin, type, length))
    assert(length ≤ len(type))
    pm_pmi_type(pm_num) = type

    /* Get next interaction */
    goto loop1
eof: continue
    close(unit = diskin)
    var_reallocb(pmi_name)
    var_reallocb(pmi_type)
    var_reallocb(pmi_sy)
    var_reallocb(pmi_reagent)
    var_reallocb(pmi_materials)
    var_reallocb(pmi_product_num)
    var_reallocb(pmi_product)
    var_reallocb(pmi_filename)

    return
end

```

See also section 1.3.

This code is used in section 1.1.

Write out data into netcdf file `pmi.nc`

⟨ Functions and Subroutines 1.2 ⟩ +≡

```

subroutine nc_write_pmi
  implicit none
  pm_common    // Common
  rf_common
  implicit none
  integer fileid

  pm_ncdecl
  nc_decls
  st_decls
  character*LINELEN description, program_version
  character*FILELEN tempfile    // local

  program_version = 'DEGAS_2_Git_commit:_%H$, _ref_names:_%d$'

  tempfile = filenames_array pmifile
  fileid = nccreate(tempfile, NC_CLOBBER, nc_stat)

  description = 'Data_for_pmi_in_degas_2'
  call ncattputc(fileid, NC_GLOBAL, 'description', NC_CHAR, string_length(description),
    description, nc_stat)

  call ncattputc(fileid, NC_GLOBAL, 'data_version', NC_CHAR, string_length(pmi_version),
    pmi_version, nc_stat)

  call ncattputc(fileid, NC_GLOBAL, 'program_version', NC_CHAR,
    string_length(program_version), program_version, nc_stat)

  pm_ncdef(fileid)
  call ncendef(fileid, nc_stat)
  pm_ncwrite(fileid)
  call ncclose(fileid, nc_stat)

  return
end

```

2 References

3 INDEX

assert: 1.2.

b: [1.2](#).

description: [1.3](#).

diskin: 1.2.

e: [1.2](#).

eof: 1.2.

file: [1.2](#).

FILE: [1](#).

fileid: [1.3](#).

FILELEN: 1.2, 1.3.

filenames_array: 1.2, 1.3.

form: 1.2.

i: [1.2](#).

implicit_none_f77: 1.2, 1.3.

implicit_none_f90: 1.2, 1.3.

int_unused: 1.2.

len: 1.2.

length: [1.2](#).

line: [1.2](#).

LINELEN: 1.2, 1.3.

loop1: 1.2.

loop2: 1.2.

ma_common: 1.2.

ma_lookup: 1.2.

NC_CHAR: 1.3.

NC_CLOBBER: 1.3.

nc_decls: 1.3.

NC_GLOBAL: 1.3.

nc_stat: 1.3.

nc_write_pmi: [1.3](#).

ncattputc: 1.3.

ncclose: 1.3.

nccreate: 1.3.

ncendef: 1.3.

next_token: 1.2.

p: [1.2](#).

parse_string: 1.2.

pm_common: 1.2, 1.3.

pm_filename: 1.2.

pm_gen: 1.2.

pm_materials: 1.2.

pm_ncdecl: 1.3.

pm_ncdef: 1.3.

pm_ncwrite: 1.3.

pm_num: 1.2.

pm_pmi_type: 1.2.

pm_product: 1.2.

pm_product_max: 1.2.

pm_product_num: 1.2.

pm_reagent: 1.2.

pm_sy: 1.2.

pm_type_len: 1.2.

pmi_filename: 1.2.

pmi_generic: 1.2.

pmi_generic_no: 1.2.

pmi_generic_yes: 1.2.

pmi_infile: 1.2.

pmi_materials: 1.2.

pmi_name: 1.2.

pmi_product: 1.2.

pmi_product_num: 1.2.

pmi_reagent: 1.2.

pmi_sy: 1.2.

pmi_type: 1.2.

pmi_version: 1.2, 1.3.

pmi_file: 1.3.

program_version: [1.3](#).

read_pmi: [1.2](#).

read_string: 1.2.

read_text: 1.2.

rf_common: 1.2, 1.3.

sp_common: 1.2.

sp_lookup: 1.2.

st_decls: 1.2, 1.3.

status: 1.2.

string_length: 1.3.

tempfile: [1.2](#), [1.3](#).

type: [1.2](#).

unit: 1.2.

var_realloca: 1.2.

var_reallocb: 1.2.

⟨ Functions and Subroutines 1.2, 1.3 ⟩ Used in section 1.1.

⟨ Memory allocation interface 0 ⟩ Used in section 1.2.

COMMAND LINE: "fweave -f -i! -W[-ybs15000 -ykw800 -ytw40000 -j -n/
/Users/dstotler/degas2/src/pmisetup.web".

WEB FILE: "/Users/dstotler/degas2/src/pmisetup.web".

CHANGE FILE: (none).

GLOBAL LANGUAGE: FORTRAN.